# GPIO, Pull-up/Pull-down & Debounce

Fundamentos de Eletrônica Digital Aplicada à ESP32

PlatformIO + ESP32 — Aula Prática

01

# Fundamentos de GPIO

Entradas, saídas, níveis lógicos e configuração na ESP32

INPUT | OUTPUT | HIGH/LOW | ESP32

# O que é GPIO?

General Purpose Input/Output — Pinos de uso geral do microcontrolador

### INPUT (Entrada)

Lê sinais externos:
botões, sensores,
chaves

### OUTPUT (Saída)

Controla dispositivos:
LEDs, relés, motores,
buzzers

### Níveis Lógicos

HIGH = 3.3V (ligado)
LOW = 0V (desligado)
Base da lógica digital

### Conceito Fundamental

GPIO é a interface fundamental entre o microcontrolador e o mundo físico. Cada pino pode ser configurado individualmente como entrada ou saída via software (pinMode).

- ⚠ Na ESP32, os GPIOs operam a 3.3V — nunca aplicar 5V diretamente!

# Configurando GPIO na ESP32

Funções essenciais: pinMode, digitalRead, digitalWrite

main.cpp

```
#include <Arduino.h>

#define BUTTON_PIN  4
#define LED_PIN    2

void setup() {
  pinMode(BUTTON_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(115200);
}

void loop() {
  int estado = digitalRead(BUTTON_PIN);
  digitalWrite(LED_PIN, estado);
}
```

**pinMode(pin, mode)**

Configura o pino como INPUT ou OUTPUT no setup()

**digitalRead(pin)**

Retorna HIGH ou LOW conforme o estado do pino

**digitalWrite(pin, val)**

Define o pino para HIGH (3.3V) ou LOW (0V)

- PlatformIO • ESP32 • Arduino Framework

# Problema: Entradas Flutuantes

O que acontece quando um pino GPIO fica sem conexão definida?

**⚠ Leitura Instável — Pino Flutuante**

```
Serial: 1 0 1 1 0 0 1 0 1 1 0 1 0 0 1
Valores aleatórios captados pelo ADC
```

### Leitura Aleatória

O pino capta ruído eletromagnético e alterna entre HIGH e LOW sem previsão

### Comportamento Errático

LEDs piscam, motores ligam sozinhos, sensores reportam valores falsos

### Impacto em Sistemas Reais

Em automação e IoT, entradas flutuantes causam falhas críticas de segurança

**Solução: Resistores Pull-up / Pull-down**
Garantem um estado lógico definido quando o botão não está pressionado, eliminando a flutuação.

- Causa #1 de bugs em projetos com botões e sensores digitais

02

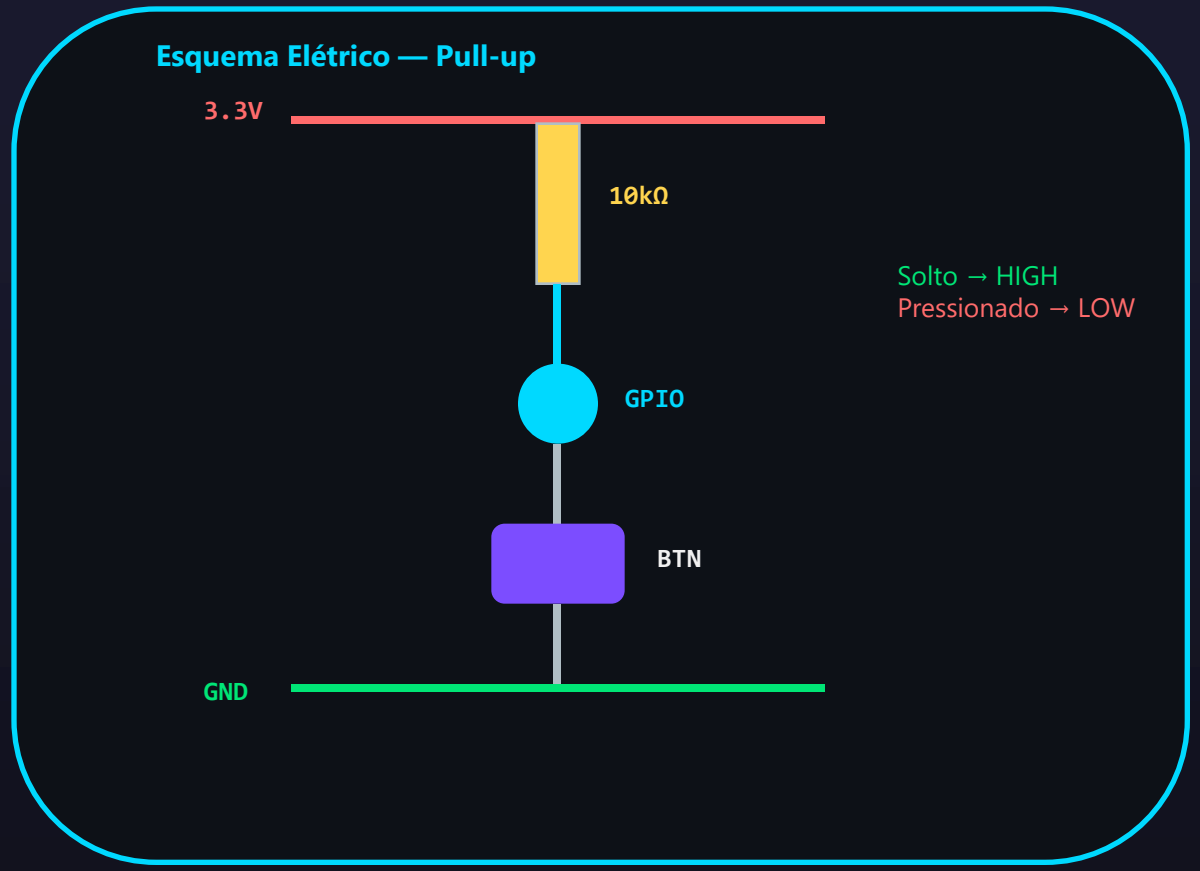
# Resistores Pull-up & Pull-down

Garantindo estados lógicos definidos nas entradas GPIO

**Pull-up** **Pull-down** **Interno** **Externo**

# Resistor Pull-up — Conceito e Circuito

Mantém o pino em HIGH quando o botão não está pressionado




## Como Funciona

O resistor conecta o pino ao VCC (3.3V). Sem pressão no botão, a corrente mantém o pino em HIGH. Ao pressionar, o pino é puxado para GND (LOW).

## Quando Usar

Ideal para botões que conectam ao GND quando pressionados. Padrão mais comum em circuitos com ESP32 e Arduino.

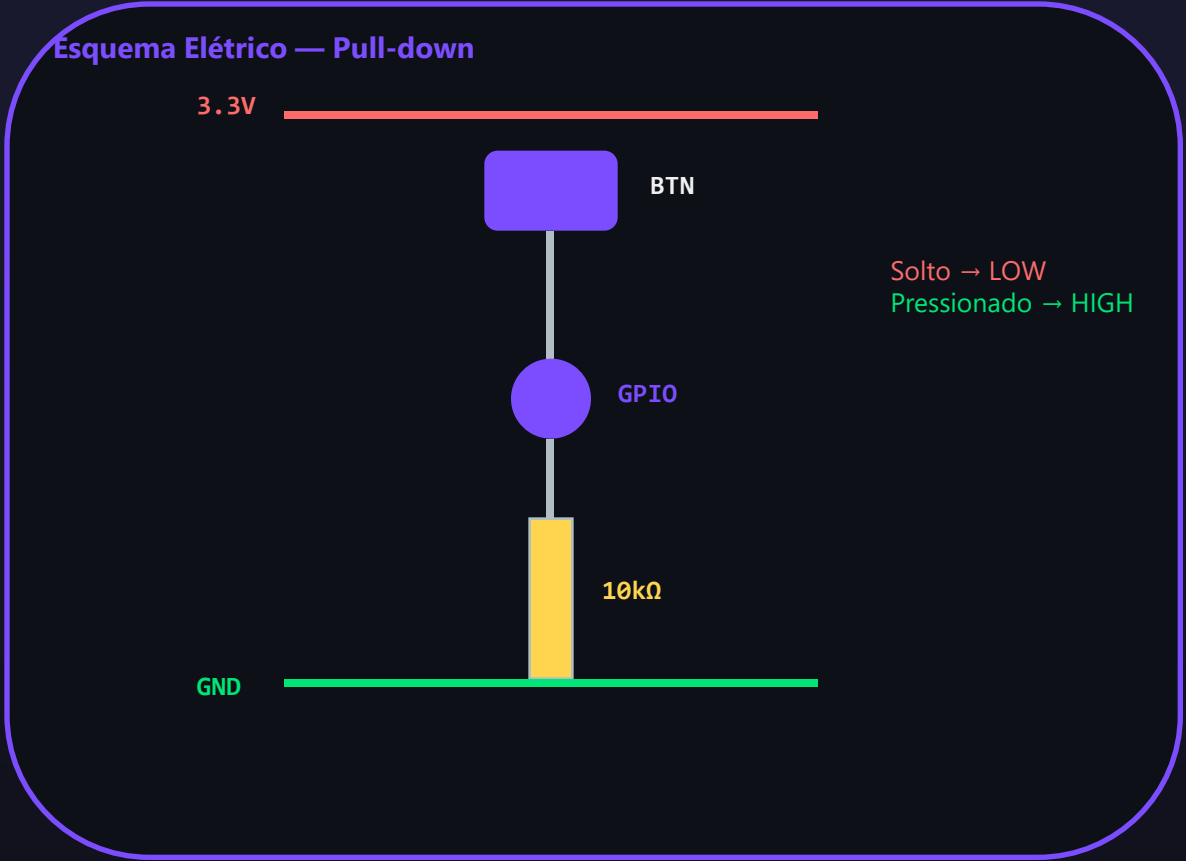
## Lógica Invertida

Atenção: botão pressionado = LOW (0), solto = HIGH (1). A lógica é inversa ao intuitivo — trate no código!

- Valor típico: 10kΩ • Consumo: ~0.33mA a 3.3V

# Resistor Pull-down — Conceito e Circuito

Mantém o pino em LOW quando o botão não está pressionado




## Como Funciona

O resistor conecta o pino ao GND. Sem pressão no botão, o pino é mantido em LOW. Ao pressionar, o pino é puxado para VCC (HIGH).

## Quando Usar

Ideal para sensores que fornecem sinal ativo em HIGH. Lógica direta: pressionado = HIGH, solto = LOW.

## Vantagem: Lógica Direta

Mais intuitivo que pull-up: botão pressionado = 1 (HIGH), botão solto = 0 (LOW). Facilita a leitura do código.

- Valor típico: 10kΩ • Corrente em repouso: ~0mA

# Comparação: Pull-up vs Pull-down + Interno vs Externo

Diferenças práticas e quando usar cada configuração

| Característica | Pull-up | Pull-down |
|---|---|---|
| **Estado padrão (solto)** | HIGH (3.3V) | LOW (0V) |
| **Ao pressionar** | LOW (0V) | HIGH (3.3V) |
| **Lógica** | Invertida | Direta |
| **Uso típico** | Botões ao GND | Sensores ativos HIGH |
| **Interno na ESP32?** | Sim (INPUT_PULLUP) | Sim (INPUT_PULLDOWN) |

### Interno (ESP32)

- Ativado via software (INPUT_PULLUP)
- ~45kΩ — suficiente para maioria dos casos
- Simplifica o circuito (sem componente extra)

### Externo (Resistor Físico)

- Valor exato (ex: 10kΩ) — maior precisão
- Necessário em cabos longos ou ambientes ruidosos
- Recomendado para produção

**Configuração na ESP32**

```
pinMode(BTN, INPUT_PULLUP);    // Pull-up interno
pinMode(BTN, INPUT_PULLDOWN);  // Pull-down interno
```

- Dica: Para prototipagem, use interno. Para produção, prefira externo.

03

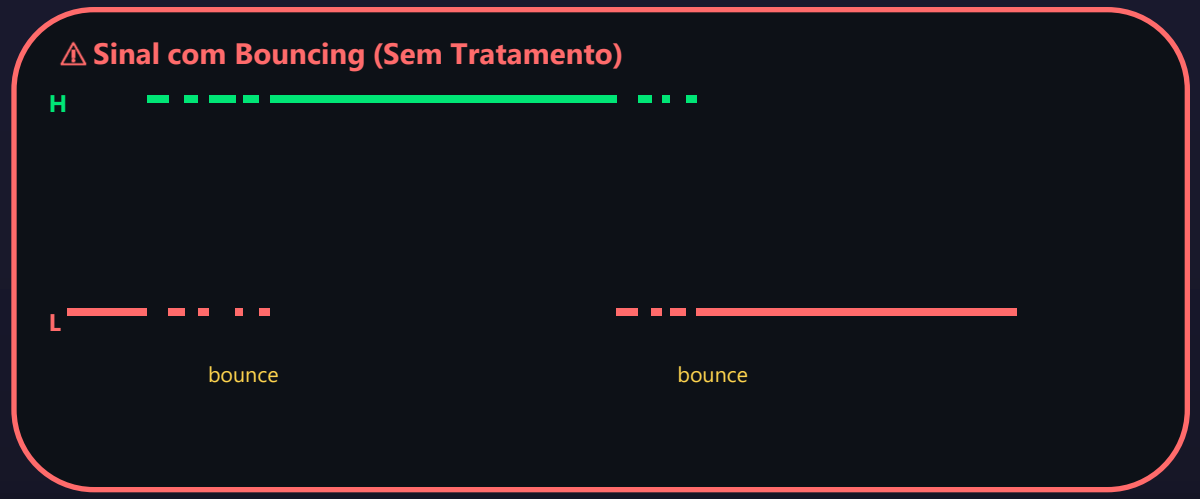
# Debounce

Tratamento de bouncing em botões e chaves mecânicas

- Bouncing
- millis()
- Filtro RC
- Hardware

# O que é Bouncing?

Problema físico de contatos mecânicos em botões e chaves



## Problema Físico

Ao pressionar um botão, os contatos metálicos vibram rapidamente (~1-5ms), gerando múltiplas transições HIGH/LOW.

## Impacto no Código

Cada bounce é interpretado como uma pressão separada. Um clique pode gerar 5-10 eventos falsos no digitalRead().

## Solução: Debounce

Técnicas de software (millis) ou hardware (capacitor RC) filtram as oscilações e entregam um sinal limpo.

- Duração típica do bounce: 1ms a 10ms

# Técnicas de Debounce + Código PlatformIO

Software com millis() e hardware com filtro RC

## 💻 Software — millis()

```
if (reading != lastState) {
  lastDebounce = millis();
}
if (millis()-lastDebounce > 50) {
  btnState = reading; // estabilizou!
}
```

## ⚡ Hardware — Filtro RC

Capacitor em paralelo com o botão:

- **R = 10kΩ** + **C = 100nF**
- τ = R × C = **1ms**
- O capacitor absorve as oscilações

**Exemplo Completo – PlatformIO (ESP32)**

```
#define BTN_PIN 15              #define DEBOUNCE_MS 50
bool btnState=HIGH, lastState=HIGH;  unsigned long lastDebounce=0;

void setup() { pinMode(BTN_PIN, INPUT_PULLUP); Serial.begin(115200); }
void loop() {
  bool reading = digitalRead(BTN_PIN);
  if(reading != lastState) lastDebounce = millis();
  if(millis()-lastDebounce > DEBOUNCE_MS) {
    if(reading!=btnState) { btnState=reading; Serial.println("Pressionado!"); }
  } lastState = reading;
}
```

- Delay de 50ms é suficiente para a maioria dos botões • Ajuste conforme necessário

# Guia Completo de Depuração

VS Code • Python • Jupyter • ESP32

- Debug é o processo de identificar, localizar e corrigir erros em software
- 
- Origem: Grace Hopper (1947) — um inseto real encontrado em um computador
- 
- Objetivo: tornar o código confiável, eficiente e livre de falhas
- 
- Competência essencial para qualquer desenvolvedor

### Erro de Sintaxe

Código mal escrito que impede a compilação

### Erro de Lógica

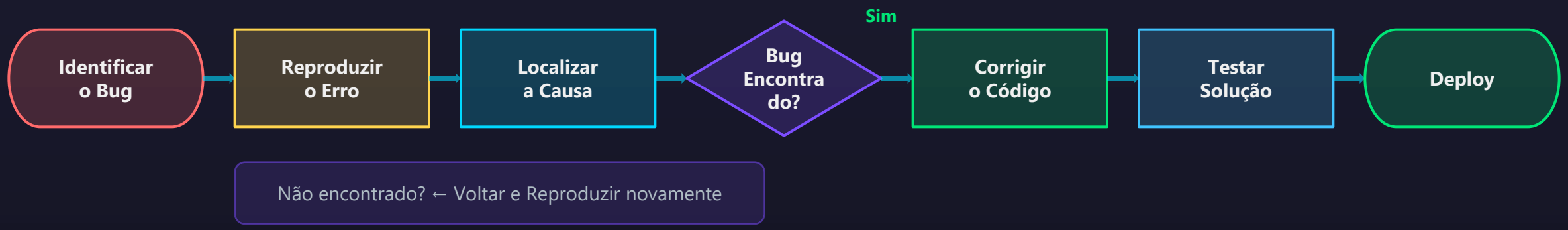
Código roda, mas produz resultados incorretos

### Erro de Runtime

Falha durante execução: exceções e crashes

### Erro de Recurso

Memory leaks, deadlocks e race conditions




## Técnicas Essenciais em Cada Etapa:

- Breakpoints e Step-through | Print/Log Debugging | Watch Variables | Call Stack Analysis
- Rubber Duck Debugging | Binary Search (bisect) | Unit Tests | Profiling

01

# Debug no Visual Studio Code

Configuração, breakpoints, watch variables e extensões

launch.json | Breakpoints | Watch | Call Stack | Extensões

# Configurando o Ambiente — launch.json

O arquivo central de configuração do debugger no VSCode

.vscode/launch.json

```
// .vscode/launch.json
{
  "version": "0.2.0",
  "configurations": [{
    "name": "Python: Current File",
    "type": "debugpy",
    "request": "launch",
    "program": "${file}",
    "console": "integratedTerminal"
  }]
}
```

**"name"**
Nome exibido no menu dropdown do debugger

**"type"**
Motor de debug utilizado (debugpy, node, cppdbg...)

**"request"**
launch = inicia o programa
attach = conecta a processo

**"program"**
Caminho do arquivo a ser depurado (variáveis aceitas)

**"console"**
Terminal para I/O do programa em execução

# Breakpoints & Controles de Navegação

**Continue**
F5
Retoma até o próximo breakpoint

**Step Over**
F10
Executa a linha sem entrar na função

**Step Into**
F11
Entra dentro da função chamada

**Step Out**
⇧F11
Sai da função atual para o caller

**Restart**
⇧⌘F5
Reinicia a sessão de debug do zero

**Stop**
⇧F5
Encerra a sessão de depuração

## Tipos de Breakpoints

● **Standard**
Ponto de parada fixo em uma linha específica

◆ **Conditional**
Para apenas quando a condição é verdadeira

◇ **Logpoint**
Loga uma mensagem sem parar a execução

⊕ **Hit Count**
Para após N execuções da mesma linha

# Watch Variables & Call Stack

## 👁 Watch Variables

Monitore expressões em tempo real durante o debug

- Adicione qualquer expressão válida
- Valor atualizado a cada step/breakpoint
- Suporta: variáveis, len(lista), obj.attr

**Exemplo de Watch:**

```
user.name       → "João"
len(items)      → 42
total > 100     → True
type(response) → <class 'dict'>
```

## 📚 Call Stack

Visualize a pilha de chamadas de funções ativas

- Mostra a sequência de chamadas até o ponto atual
- Clique em qualquer frame para navegar
- Essencial para entender recursão e callbacks

**Exemplo de Stack:**

```
→ validate_input  :42
  process_data    :28
  handle_request  :15
  main            :5
```

# Extensões Essenciais para Debug

## Python Debugger

Essencial

Debugpy integrado ao VSCode. Breakpoints, watch, call stack para Python nativamente.

## C/C++

Embarcado

GDB/LLDB integrado. Indispensável para debug de firmware embarcado.

## PlatformIO

ESP32

IDE completa para IoT. Debug ESP32 com GDB via JTAG ou USB.

## Error Lens

Visual

Mostra erros inline no editor com cores e destaque. Feedback visual imediato.

## GitLens

Histórico

Blame annotations inline. Descubra quem e quando alterou o código problemático.

## Thunder Client

API

Teste APIs sem sair do VSCode. Debug de respostas HTTP em tempo real.

02

# Debug em Python

PDB, debugpy, logging e casos práticos

pdb | debugpy | logging | Casos Práticos

# pdb — O Debugger Nativo do Python

```python
# Inserir breakpoint no código
import pdb

def calcular_media(notas):
    total = sum(notas)
    pdb.set_trace()  # breakpoint!
    media = total / len(notas)
    return media

# Python 3.7+: use breakpoint()
```

## Comandos Essenciais do PDB

- **n (next)** → Executa a próxima linha
- **s (step)** → Entra dentro da função
- **c (continue)** → Continua até o próximo breakpoint
- **p var** → Imprime o valor da variável
- **l (list)** → Mostra o código ao redor
- **q (quit)** → Sai do debugger

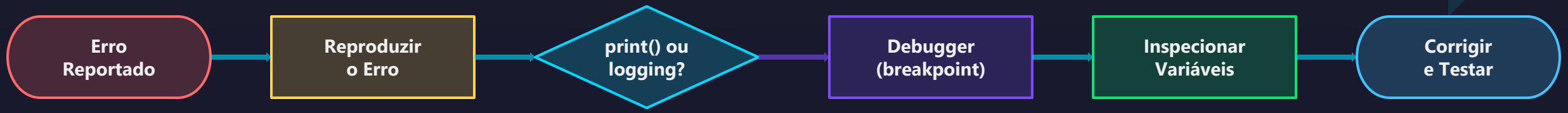
💡 **Dica: Python 3.7+ simplifica tudo**

Use `breakpoint()` em vez de `import pdb; pdb.set_trace()`. Controle via `PYTHONBREAKPOINT` env var.

# Logging vs Debugging

| Aspecto | 📋 Logging | 🔍 Debugging |
|---|---|---|
| Quando usar | Monitoramento contínuo em produção | Investigação ativa em desenvolvimento |
| Pausa execução? | Não — executa normalmente | Sim — para nos breakpoints |
| Granularidade | Níveis: DEBUG, INFO, WARNING, ERROR, CRITICAL | Inspeção completa: variáveis, stack, memória |
| Overhead | Mínimo — pode ficar em produção | Alto — apenas em ambiente de dev |
| Persistência | Logs salvos em arquivos/serviços | Sessão temporária — nada é salvo |

# Fluxo de Debug em Python — Caso Prático




## Caso 1: ZeroDivisionError

Função recebe lista vazia
→ breakpoint() antes da divisão
→ Inspecionar len(lista) no Watch
→ Adicionar validação de input

## Caso 2: KeyError em Dict

API retorna JSON sem campo esperado
→ Conditional breakpoint: key not in data
→ Verificar response.status_code
→ Usar dict.get() com default

## Caso 3: Loop Infinito

While loop nunca termina
→ Breakpoint dentro do loop
→ Watch na variável de controle
→ Step through para ver mutações

03

# Debug com ESP32

Serial, JTAG, USB nativo e configuração completa

Serial | JTAG | USB Nativo | PlatformIO | ESP-IDF

# Serial Monitor vs JTAG Debug

## Debug via Serial (printf-style)

- Sem hardware adicional — usa UART nativo
- Baseado em Serial.println()
- Não para a execução do programa
- Impacta timing — pode mascarar bugs
- Sem inspeção de memória/registradores

**Ideal para: prototipagem rápida, logs simples**

## Debug Real via JTAG/SWD

- Breakpoints reais — pausa o processador
- Step-through de código C/C++
- Inspeção de memória e registradores
- Watch expressions em tempo real
- Requer hardware: FTDI ou adapter JTAG

**Ideal para: bugs complexos, timing, produção**

**Hardware para JTAG no ESP32 tradicional:** ESP-Prog (oficial Espressif) | FTDI FT2232HL | J-Link EDU Mini | Olimex ARM-USB-OCD-H

# Arquitetura de Debug — ESP32 + JTAG

### 🖥 PC / VSCode

GDB Client
PlatformIO / ESP-IDF
OpenOCD Server

USB →

### 🔌 JTAG Adapter

ESP-Prog / FTDI
Converte USB → JTAG
Protocolo: IEEE 1149.1

JTAG →

### ⚙ ESP32

CPU Xtensa/RISC-V
On-Chip Debug Module
2 hardware breakpoints

### 📌 JTAG Pinout — ESP32

| | | |
|---|---|---|
| TDI | → GPIO12 | (Test Data In) |
| TDO | → GPIO15 | (Test Data Out) |
| TCK | → GPIO13 | (Test Clock) |
| TMS | → GPIO14 | (Test Mode Select) |

### 🔧 Stack de Software

- VSCode + PlatformIO Extension
- GDB (xtensa-esp32-elf-gdb)
- OpenOCD (bridge GDB ↔ JTAG)
- ESP-IDF / Arduino Framework
- FreeRTOS task-aware debugging

# ESP32-S3 / C3 — Debug sem Hardware Externo

🚀 **USB nativo integrado = JTAG via cabo USB — sem adaptador externo!**

## ESP32-S3

Xtensa LX7 (Dual-Core)

USB OTG nativo
Wi-Fi + BLE 5.0
Até 16MB Flash
AI acceleration (vector)

## ESP32-C3

RISC-V (Single-Core)

USB Serial/JTAG nativo
Wi-Fi + BLE 5.0
Até 4MB Flash
Menor custo e consumo

## Fluxo de Configuração

1. Instalar ESP-IDF/PIO → 2. Selecionar board S3/C3 → 3. Conectar via USB → 4. Configurar OpenOCD → 5. Debug! F5 no VSCode

04

# Debug no Jupyter Notebook

Debug interativo, inspeção visual e ferramentas integradas

%debug | ipdb | Visual | Jupyter vs IDE

# Ferramentas de Debug no Jupyter

## %debug / %pdb

Magic commands do IPython.
%debug ativa post-mortem após erro.
%pdb on ativa debugger automático.
Interface pdb no output da célula.

## ipdb (IPython Debugger)

pip install ipdb
import ipdb; ipdb.set_trace()
Autocompletar e syntax highlighting.
Melhor UX que pdb padrão.

## Debugger Visual (JupyterLab)

JupyterLab 3.0+ tem debugger nativo.
Breakpoints visuais nas células.
Painel de variáveis integrado.
Xeus-python como kernel.

## PixieDebugger

Debug visual para Jupyter clássico.
Interface gráfica com breakpoints.
Inspeção de DataFrames inline.
pip install pixiedust

# Quando usar Jupyter vs IDE Tradicional

## 📓 Use Jupyter quando...

- ✓ Exploração de dados e prototipagem rápida
- ✓ Visualização inline (matplotlib, seaborn)
- ✓ Debug célula por célula (isolamento natural)
- ✓ Machine Learning e Data Science
- ✓ Documentação junto com o código

## 📄 Use IDE quando...

- ✓ Projetos com múltiplos arquivos e módulos
- ✓ Debug complexo (conditional breakpoints)
- ✓ Refactoring e navegação em código grande
- ✓ Git integrado e versionamento
- ✓ Deploy e CI/CD integrados

💡 **Melhor abordagem: combine os dois!**

Prototipar no Jupyter → Refatorar na IDE → Testar no Jupyter → Deploy via IDE. VSCode abre notebooks nativamente (extensão Jupyter), unificando o workflow em um só ambiente.

# Matriz Comparativa — Todos os Ambientes

| Recurso | VSCode | Python/pdb | ESP32 | Jupyter |
|---|---|---|---|---|
| **Breakpoints** | Visual + condicional | pdb.set_trace() | JTAG hardware | %debug / ipdb |
| **Step-through** | F10/F11 visual | n, s, c no terminal | GDB step/next | Célula por célula |
| **Watch Vars** | Painel dedicado | p variável | GDB watch | display() / print() |
| **Call Stack** | Painel visual | where / bt | backtrace GDB | Traceback nativo |
| **Facilidade** | ★★★★★ | ★★★☆☆ | ★★☆☆☆ | ★★★★☆ |
| **Profundidade** | ★★★★☆ | ★★★☆☆ | ★★★★★ | ★★☆☆☆ |